

Aula 5 — Maestro E2E em RN + Lab Perf/Security + Trabalho Final

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 02/07/2026

Agenda de hoje (3h30)

1. **Plantão de setup + Lab perf/security** — rodam em paralelo (~40min)
2. Recap da pirâmide de testes
3. **Maestro sobre o app RN** — fechar os 5 flows (Atividade 4)
4. Intervalo
5. **Trabalho Final** — revelar temas + rubrica
6. Combinar a data da apresentação

Como vamos usar os próximos ~40min

Ainda ajustando o Android Studio/emulador? Chama o prof — resolvemos juntos, com calma.

Device/emulador já de pé? Começa o **lab de perf/security**, sozinho:

`exercicios/03-maestro-e2e/lab-perf-security.md`

Sem device ainda? Começa pela parte do lab que **não precisa de adb** — achar e corrigir o manifest.

| Todo mundo com algo pra fazer em paralelo. Reconvergimos em ~40min.

O app-alvo: CineFav

`com.puciec.cinefav` — abre na tela de **Login**.

Login mock: **qualquer email com @ + senha 4+ caracteres**.

Ex.: `aluno@puc.br` / `1234`.

Dados **mockados** por padrão (sem token TMDB, sem rede) — flows **determinísticos**.

Matrix sempre existe.

Lab — Performance + Security no CineFav

Self-guided. Sem slide teórico genérico — medindo o app que vocês já têm rodando.

Performance — cold start (quem tem device/emulador)

```
adb shell am force-stop com.puciec.cinefav  
adb shell am start -W com.puciec.cinefav/.MainActivity
```

Olhar **TotalTime** (ms). Rodar 3x.

Depois **sem** o **force-stop** (warm start) — comparar.

Performance — jank ao rolar (quem tem device/emulador)

Rolar a lista de filmes no device, depois:

```
adb shell dumpsys gfxinfo com.puciec.cinefav
```

Olhar **Janky frames** (% acima de 16ms/60fps).

Security — achado real no manifest (todo mundo, sem adb)

```
cd exercicios/03-maestro-e2e/pratica
cat android/app/src/main/AndroidManifest.xml \
  | grep -E "allowBackup|EXTERNAL_STORAGE|SYSTEM_ALERT"
```

Dois achados reais (não plantados):

1. `allowBackup="true"` — extrai dados do app via `adb backup` sem root (OWASP M9/M2, MASVS-STORAGE)
2. `READ/WRITE_EXTERNAL_STORAGE` + `SYSTEM_ALERT_WINDOW` — permissões que o app não usa

Security — fix (todo mundo — rebuild não precisa de device)

```
- android:allowBackup="true"  
+ android:allowBackup="false"
```

- remover as 3 permissões não usadas.

```
cd android && ./gradlew assembleDebug  
adb install -r app/build/outputs/apk/debug/app-debug.apk # só quem tem device
```

Achar → corrigir → reverificar. É literalmente o que MASVS/OWASP Mobile Top 10 pedem — vocês acabaram de fazer isso de verdade, sozinhos.

Recap — pirâmide de testes no CineFav

Camada	O que testamos	Como	Status
Unit	funções puras, stores Zustand	Jest, sem simulador	✓
Integração	fluxo entre componentes (RNTL, DOM emulado)	Jest + RNTL, mock	✓
E2E	app de verdade, abrindo no device	Maestro	agora

| Agora é a única camada que abre o app **real** — sem emular nada.

testIDs — a ponte entre RNTL e Maestro

No componente RN:

```
<Pressable testID={`movie-card-${movie.id}`} onPress={...}>
```

Vocês já usaram esse **mesmo testID** na Atividade 2/3 (RNTL):

```
screen.getByTestId('movie-card-603')
```

No Maestro:

```
- tapOn:  
  id: "movie-card-603"
```

Mesmo testID, três camadas. Isso não é coincidência — é a instrumentação de acessibilidade/teste do componente RN sendo reaproveitada.

tapOn: id: vs tapOn: "texto"

```
# Frágil – quebra se o texto mudar (il8n, dado dinâmico)
- tapOn: "Matrix"

# Estável – aponta pro elemento, não pro conteúdo
- tapOn:
  id: "movie-card-603"
```

Mesmo argumento de `getByTestId` vs `getByText` no RNTL. Prefira ID quando disponível.

Accessibility role — o mesmo elemento, duas lentes

```
- tapOn:  
  id: "movie-card-heart-603"
```

Testa que o elemento existe. Mas o **leitor de tela** de um usuário cego não vê ID — vê:

```
accessibilityRole="button"  
accessibilityLabel="Adicionar favorito"
```

Se o componente RN não tiver esses props, o teste de ID passa mas a **acessibilidade real falha**.

Testar por ID garante que existe; testar por role/label garante que é **usável**.

Os 5 flows da Atividade 4

Flow	Testa	Conceito novo
01-launch.yaml	login + lista aparece	modelo resolvido ✓
02-search.yaml	buscar filme	env (SEARCH_TERM)
03-favorite.yaml	favoritar → conferir em Favoritos	assert cross-tela
04-detail.yaml	abrir detalhe, favoritar, voltar	navegação
05-js-dynamic.yaml	termo de busca via JS	evalScript

01 já resolvido — modelo. 02 fazemos juntos. 03 – 05 + bônus fragment: solo, em casa.

Rodando

```
cd exercicios/03-maestro-e2e/pratica  
maestro test flows/01-launch.yaml  
maestro studio          # editor visual - localhost:9999
```

Celular físico? Pule `maestro-local.sh` (ele tenta bootar emulador) — vá direto no `maestro test`.

E2E é caro e às vezes flaky

Quando falha, o Maestro mostra exatamente onde:

```
x Unable to find element with text: "Spider-Man"
```

Rodar de novo — às vezes passa (flake real, aconteceu na última aula). **Retry existe por isso.** Diferente de unit/integração (determinístico), E2E depende do device/emulador de verdade.

Trabalho Final

60 pts · grupo de 3–4 · 1 dos 10 temas

Os 10 temas

#	Tema
1	Automação Mobile E2E (Maestro + Cloud Devices)
2	Native UI Testing (Espresso + XCUITest)
3	Performance Mobile Testing — aprofunda o lab de hoje
4	Mobile Security Testing — aprofunda o lab de hoje
5	Visual AI em Mobile (Appliteools/Percy)
6	Test Generation com LLM (Claude API + Maestro)
7	AI Agent Exploratory (AppAgent/DroidBot+LLM)
8	CI/CD Pipeline Mobile (Actions + Fastlane + Firebase)
9	Mobile API Contract Testing (Pact + Maestro)
10	Mobile API Mocking (WireMock + Mockito)

Enunciado completo: `exercicios/04-trabalho-final/enunciado.md` (público, no repo).

Rubrica (60 pts)

Critério	Pts
Apresentação oral (10min + 5min Q&A)	12
Artefato técnico (repo funcional)	21
Relatório analítico (1pg)	12
Domínio conceitual em Q&A	9
Originalidade e profundidade	6

Entrega: 15/07/2026 (já no Canvas). **Data da apresentação: combinamos agora.**

Combinando a data da apresentação

Aula final = apresentações + IA em testes/CI-CD (conteúdo remanescente).

Falta só isso.